Finding Belief Geometries with SAEs

Intro

The primary goal is to explore whether it is possible to use SAEs to find the true belief geometries of component processes responsible for generating sequences on which a transformer has been trained. At a high level, the approach explored here can be described with these steps:

1) Generate sequences with a multipartite (MP) model with components made of hidden Markov models (HMMs) and generalized HMMs (GHMMs)
2) Fit a transformer to predict next token/emission from the multipartite model
3) Fit top k and vanilla L1 SAEs
4) Soft cluster active latents, currently with k-subspace clustering
5) Find candidate belief geometries using Gromov-Wasserstein optimal transport distance
6) Refine clusters by removing latents that contribute distortion to optimal transport to all three top geometries above some threshold
7) Refit filtered clusters to belief geometries

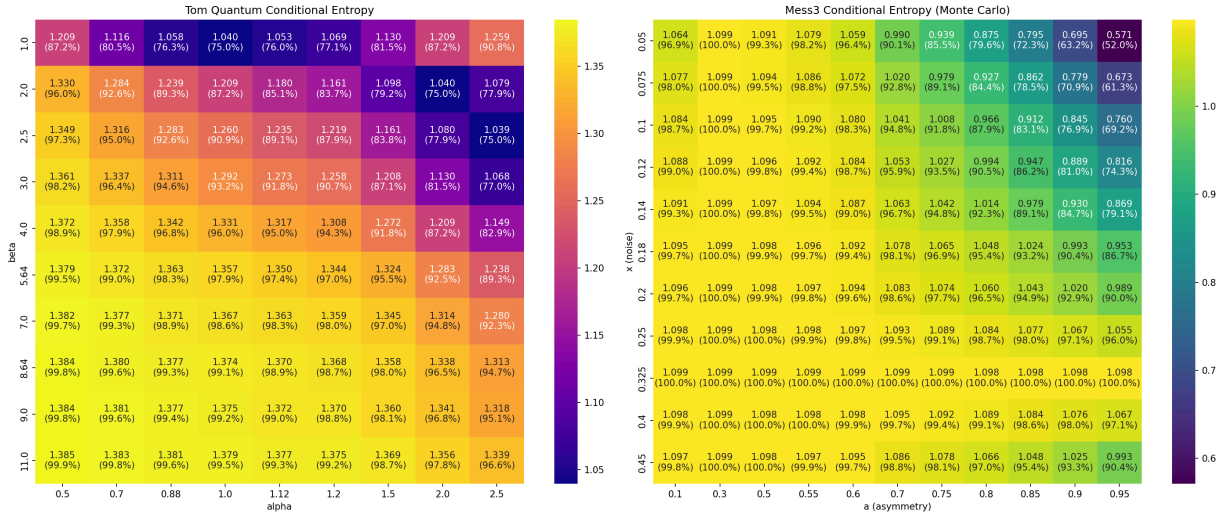
Generator Model

Our generating model is a multipartite process composed of five independent models, three hidden Markov models (HMMs) and two generalized HMMs (GHMMs). The three HMMs are Mess3 processes (Marzen and Crutchfield). The two GHMMs are Bloch Walks (Riechers, Elliot, Shai). These are the parameterizations.

```
"3xmess3_2xtquant_003": [
    {"type": "mess3",
        "instances": [
            {"x": 0.05, "a": 0.85},
            {"x": 0.075, "a": 0.9},
            {"x": 0.1, "a": 0.95}
        ]},
    {"type": "tom_quantum",
        "instances": [
            {"alpha": 1.51, "beta": 3.07},
            {"alpha": 1.99, "beta": 2.51}
         ]}]
```

These were chosen with an eye towards choosing parameterizations with a reasonable theoretical conditional entropy. Note that if conditional entropy, the entropy of the next token given knowledge of the past, is too close to the uniform entropy over the vocabulary, then that means the conditional distribution is nearly uniform over the vocabulary and it is impossible to

learn to predict the process well. The figures below give the conditional entropy as a raw value and as the percent of uniform entropy over the vocabulary.



The full multipartite (MP) model is created by independently generating emissions from all of the component models for each iteration. Then the emission of the MP model is the Cartesian product of the emissions from the components. In particular, the Mess3 HMM has a vocabulary of three tokens and the Bloch Walk has a vocabulary of four tokens. So our MP has `4 * 4 * 3 * 3 * 3 = 432` tokens in its vocabulary, one for each possible combination of tokens from the five component models.

Transformer Model

Setup and Training

Next we fit a small TransformerLens model to predict the next emission from our MP given the previous emissions in our context. The transformer is parameterized as:

```
--d_model 128
--n_heads 4
--n_layers 3
--n_ctx 16
--d_head 32
--act_fn "relu"
```

This gives a total of 709k trainable parameters in our transformer. We train the model on sequences of length 16 in batches of size 2,048 from the MP generator described above. Optimization is on next-token cross-entropy using AdamW. The base learning rate is 1e-4 with betas (0.9, 0.95) and weight decay 0.01. Gradients can optionally be clipped; by default we leave clipping off because the small network stays stable. We use cosine decay on the learning rate with a linear warmup of 2,000 steps, finishing at 10% of the base learning rate. Once we hit

the minimum iterations, we calculate a moving average error (EMA) with beta = 0.98. We stop early if EMA fails to improve by 1e-7 for 30 iterations. We then choose the step with the lowest mean error out of the last 30. Using this training regime the MP model stopped after 6,000 training steps.

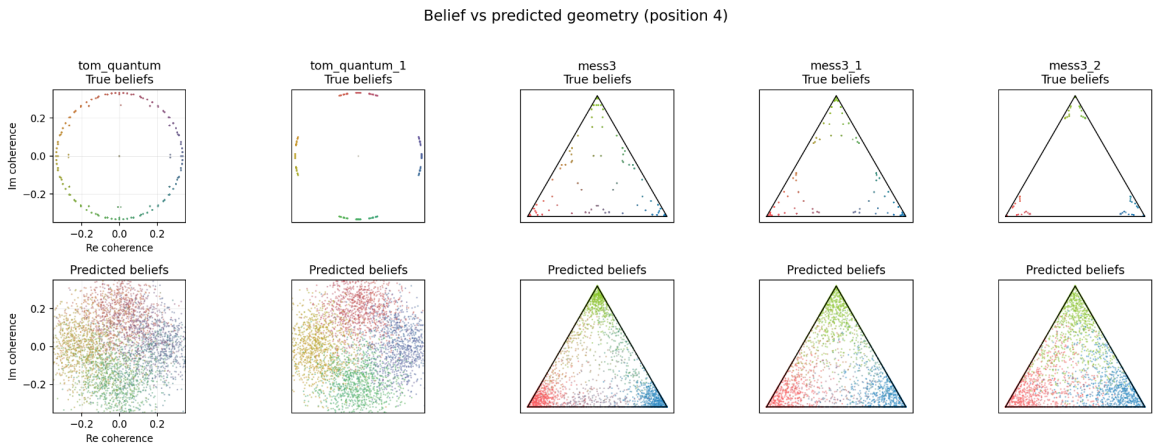
Performance

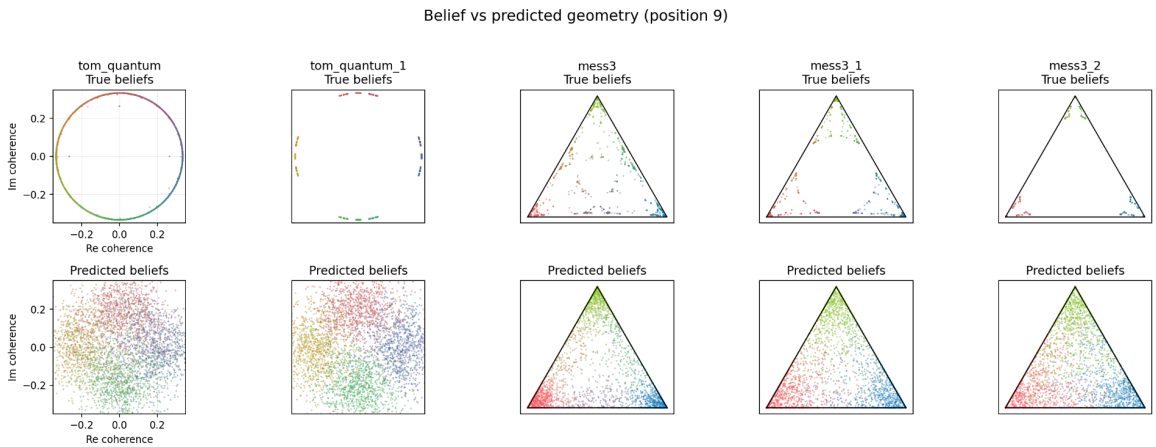
Given what we noted above about conditional entropy, unsurprisingly the model does not perform well predicting the next token. The model we use in this work is listed here as MP Model 2. MP Model 1 was an earlier generator.

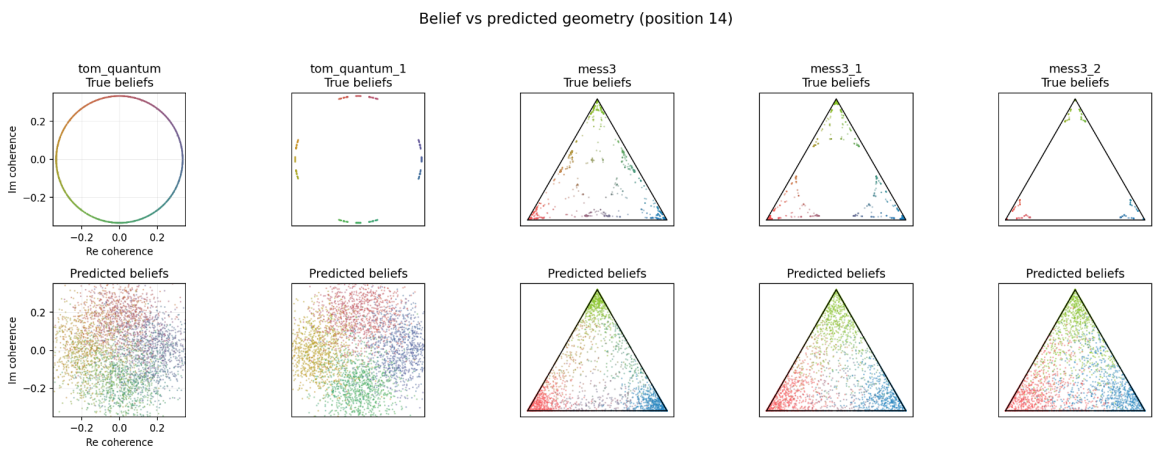
	MP Model 1	*MP Model 2*
Top-1 Accuracy	0.61%	7.1%
Top-5 Accuracy	3.0%	24.1%
Top-10 Accuracy	5.6%	35.9%
Cond. Entropy	5.95 Nats	4.62 Nats
Uniform Entropy	6.07 Nats	6.07 Nats

Representations

Despite the difficulty of prediction, the transformers still learn decent representations of the generator components! The PCA projections that best capture the generator component belief geometries are shown at the end of this doc. Below we see belief states at the 5th, 10th, and 15th tokens in a sequence. The top row of each figure is the ground truth belief states. Note that even by the 15th position, the components haven't fully explored the reachable portions of their belief geometry. The bottom row is the predicted belief state for each component. These are linear predictions from the residual activations at layer 3 – the final layer – of the model. The points are colored by their true belief state. As in, the point color is the mixture of the colors assigned to the pure states, weighted by the level of belief. For example, if a ground truth belief state of a Mess3 process is (0.2, 0.2, 0.6) for hidden states 0, 1, and 2 respectively, then a point will be drawn at (0.2, 0.2, 0.6) and colored with RGB values proportional to that vector. This coloring scheme is true for predicted belief states as well. So if the belief state was estimated by a linear projection from the activation vector to be (0.1, 0.4, 0.4), then the color with RGB values (0.2, 0.2, 0.6) would be drawn at point (0.1, 0.4, 0.4), and then projected down to 2D for the visualization.







SAEs

The primary goal of this work is to understand whether, when we know the ground truth belief states, SAEs can be used as a tool for detecting the shape of those belief geometries. We are also interested in the question of whether SAEs can assist in detecting when different generator

components are active. But in our current toy model all of the components are active for every token, this is not a setup for investigating that question.

We examine a range of SAEs. We have two goals: 1) finding the best performing version and parameterization for the current problem; 2) gaining some insights into the general performance of SAEs in the context of belief geometries.

We have fit a range of SAEs to study on our current toy MP model. We have fit top K SAEs with K values of: 3, 4, 5, 6, 7, 8, 10, 12, 14, 16, 19, 22, 25. We have fit vanilla SAEs – where the objective function is reconstruction L2 loss plus a parameter lambda times the L1 norm of the latents – with lambda values: 1e-3, 5e-3, 1e-2, 2e-2, 3e-2, 4e-2, 5e-2, 6e-2, 7e-2, 8e-2, 9e-2, 0.1, 0.125, and 0.15.

Clustering

Now that we have estimated our SAE latents, we want to see if they've learned to represent the belief geometries of our generator components. The first step is to cluster the latents. Inspired by the paper [The Geometry of Concepts: Sparse Autoencoder Feature Structure](), we first tried spectral clustering with both cosine similarity and Euclidean distance as the metrics. This did not work well in our case. We will revisit clustering methodology when we graduate from toy models to LLMs. Currently, we haven't found a clustering method yet that achieves our objective. The best working model so far is a variant of [k-subspace clustering](), where we initialized clusters with [QR column-pivoting]() but on the transposed, raw latent direction matrix. In the original paper they decompose the eigenvectors of an adjacency matrix. We motivate decomposing the raw matrix with the QR methodology itself. We are looking to find near-orthogonal subspaces that the model has learned to use to represent the belief states of the generator components. QR decomposition proceeds exactly by trying to find the next vector that is most orthogonal to those already chosen. The original k-subspaces algorithm leverages random seeding, which does not line up well with our motivation. You can think of the clustering algorithm like:

1) Column normalize the transpose of the SAE decoder matrix W_d^T which is shape $(d_{\text{model}}, d_{\text{sae}})$.
2) Set K. Currently this is either done manually or with a heuristic like $K = \min(\sqrt{d_{\text{sae}}, \max_{\text{clusters}}})$
3) Do QR decomp up to K pivots, which gives you the K rows from W_d that are most orthogonal to each other and will serve as the cluster seeds
4) Note that because we normalized the decoder directions, the seed directions are an orthonormal basis for the 1D space they themselves span
5) Assign the rest of the decoder directions to the seeded clusters by projecting each to all the seeds and assigning it to the one that leaves the smallest residual. Iterate through the following until no rows change clusters:
    a) Project every non-seed direction to all clusters using the current orthonormal bases for each cluster: $\hat{x_j}^{(k)} = B_k(B_k^T x_j)$

b) (Potentially re-)assign x_j to cluster k where $k = \min_k ||\hat{x_j^{(k)}} - x_j||$
c) For any clusters with a new membership, whether through addition or deletion, estimate a new orthonormal basis using SVD. We choose the rank of the subspace by:
    i) Choose the smallest r where the variance explained through r dimensions is greater than a variance threshold (ranges 0.8 to 0.95). This is the first candidate rank
    ii) Look at the ratios between successive singular values. The first ratio exceeding a threshold (have looked in the 1.5 to 3 range) is another candidate value of r
    iii) Take the minimum of those two candidates

Belief Geometries

AAnet: AE Projection to Simplex

This is an alternative to the original Gromov-Wasserstein analysis described below. Unfortunately we have similar issues, with higher dimensional simplices fitting better for the cases we've tried so far, even for just a single Mess3 generator. Current hypothesis is that there's just too much multidimensional noise in the representations learned by the model, so it's just true that you lose information when you squash down to a 2-simplex. To say it another way, with the first three PCs we get a pretty triangle draped over a hemisphere. Those three PCs represent 85-90% of the variance, but not all of it! So it's in fact true that the representation isn't super close to a 2D manifold.

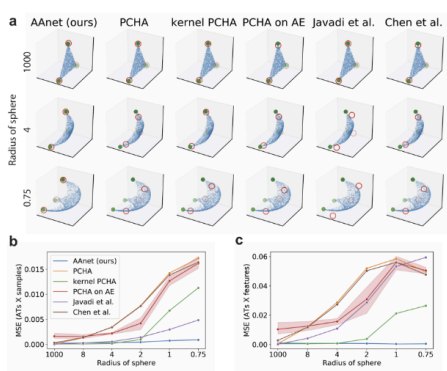


Fig. 2. (**a**) Points uniformly distributed within a triangle (blue dots) are projected onto a sphere of varying radius (columns). 6 AA methods are compared on their ability to recover the vertices of the triangle (green dots) and learn the correct mixture of archetypes for each point. Red circles mark the recovered archetypes. Right, the MSE between ground truth and recovered archetypal spaces (**b**) and recovered archetypes (**c**) are displayed for each method. Shaded area marks 95% CI over 5 runs.

This method comes from this paper.You can see a figure from that paper here, where they use as the first demo the case of uniform points over a triangle projected on spheres of different radii! The basic idea is super simple. They fit an autoencoder to input samples. The bottleneck layer is the dimension of the number of vertices in the simplex they're projecting to. The objective function is a combination of the typical reconstruction error and a set of penalties designed to steer the projections to the K dimensions in the bottleneck layer towards projecting into a $(K-1)$-simplex.

More formally, let a be an activation vector of length d_{model}. Our autoencoder consists of an encoder E that learns a non-linear projection from d_{model} to K dimensions, and a decoder D that learns a non-linear projection from K to d_{model} dimensions. Our primary loss is our reconstruction loss given by $||a - D(E(a))||_2$. To enforce not just a K dimensional projection but a projection into a simplex, we need to enforce that all the elements of the bottleneck vector are positive and sum to one.

Equality constraints can be difficult for automatic optimizers. So this is achieved in the following way. The model actually projects to $K-1$ dimensions rather than K. Let x be the full bottleneck projection with K dimensions. Then the model actually projects to $x' = E'(a)$ with $K-1$ dimensions. Then the final element of x is set to $1 - ||x'||_1)$. To push towards a simplex representation, two penalties are added: $\lambda_1(1 - ||E'(a)||_1)$, and $\lambda_2 \sum_i |x_i| \mathbf{1}(x_i < 0)$.

With this approach we can fit a set of activations to a $(K-1)\text{-simplex}$. Results so far are just okay. On a MP model 2 as described above we never get the 2-simplex as the start of the plateau in reconstruction error. Sometimes the reconstruction error declines further even past the 3-simplex. We then also fit the AAnet on activations from transformers trained just on different parameterizations of the Mess3. There the 3-simplex is again usually the minimum reconstruction error, though there's a smaller drop from 2-simplex to 3-simplex and sometimes the error rises after the 4-simplex.

Gromov-Wasserstein

This method ended up not performing great on first pass. Whether we could massage it and get better performance with something like an elbow plot analysis is on the to do list.

We model each SAE latent cluster as a finite metric space $(\mathcal{X}, d_{\mathcal{X}})$ built from the latent directions in that cluster and a pairwise similarity metric (currently cosine and Euclidean are equivalent because the directions are L2 normalized). Each candidate belief geometry, such as a circle or 2-simplex, is represented by a finite metric space $(\mathcal{Y}, d_{\mathcal{Y}})$ obtained by sampling anchor points from the geometry's uniform distribution and inheriting the metric $d_{\mathcal{X}}$.

To align a latent cluster with a geometry we solve a Gromov–Wasserstein (GW) optimal transport problem. Intuitively, we look for a soft matching γ between latent points and geometry anchor points that preserves pairwise distances as much as possible. The GW objective is

$$\mathrm{GW}(\mathcal{X}, \mathcal{Y}) = \min_{\gamma \in \Pi(p,q)} \sum_{i,j,k,\ell} \Big| d_{\mathcal{X}}(x_i, x_k) - d_{\mathcal{Y}}(y_j, y_\ell) \Big|^2 \gamma_{ij} \gamma_{k\ell}$$

where $\Pi(p,q)$ is the set of couplings with prescribed marginals p and q, which in our case are uniform so equal to $1/|\mathcal{X}|$ and $1/|\mathcal{Y}|$ respectively. The double sum accumulates the distortion, the increase in the pairwise measure between two points after they are mapped into the new

space via γ. A perfect isometry has zero distortion; higher values signal that the latent cluster does not live on that geometry.

Algorithmically, we use the entropically regularized variant solved by alternating updates.

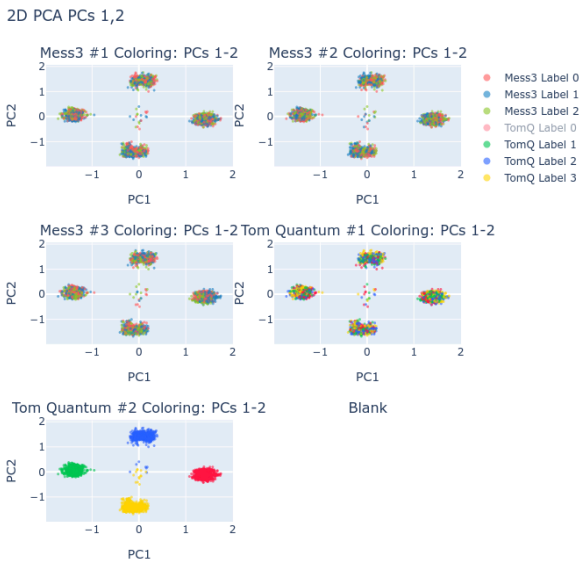
You can think of the GW distance above as the sum of all pairwise distortions, weighted by the matchings γ_{ij} and $\gamma_{k\ell}$.

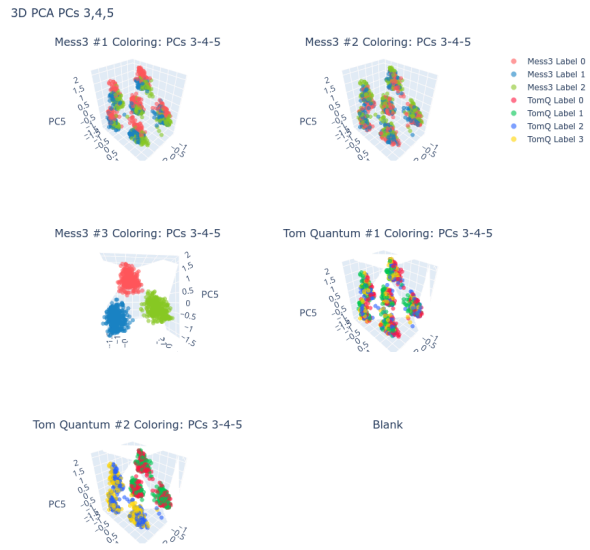
When you run an entropic GW solver (Sinkhorn type), each iteration fixes the current coupling (γ) and constructs a cost matrix (C) for the next OT update.

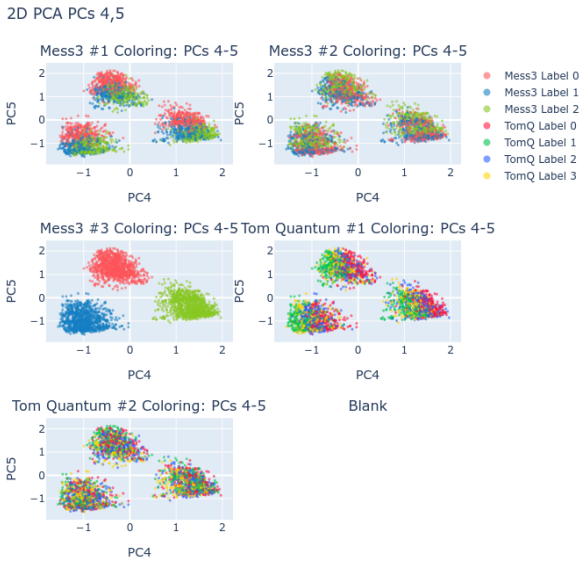
1) Keep the current (γ) from the previous iteration in the second slot (indices (k, ℓ)).
2) For each latent–geometry pair ((i,j)), compute
   $C_{ij} = \sum_{k,\ell} \left| d_{\mathcal{X}}(x_i, x_k) - d_{\mathcal{Y}}(y_j, y_\ell) \right|^2 \gamma_{k\ell}$
   This is just the GW distortion summed over all latent pairs (i, k) and geometry pairs (j, ℓ). How costly would it be to match x_i with y_j given the rest of the coupling?
3) Once you have (C), you solve a regular entropic optimal transport problem:
   $\gamma^{\text{new}} = \operatorname{argmin}_{\gamma \in \Pi(p,q)} \sum_{i,j} C_{ij}\gamma_{ij} + \varepsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$
   This is the Sinkhorn step: minimize linear cost plus entropy, subject to the fixed marginals.
4) After you get γ^{new}, plug it back into the distortion formula, recompute (C), and repeat until convergence.

PCAs

Layer 2

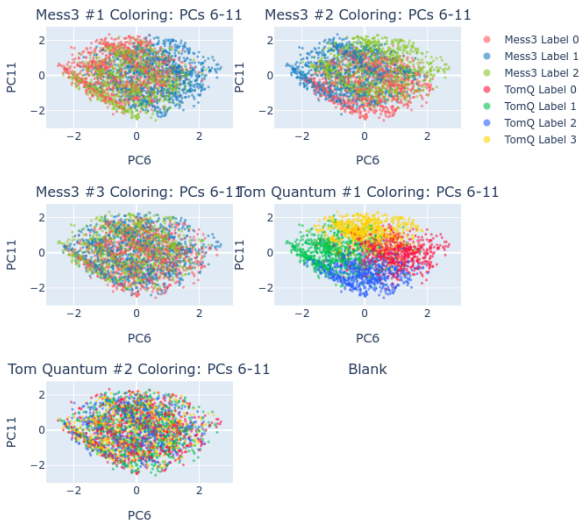

2D PCA PCs 1,2
Mess3 #1 Coloring: PCs 1-2
Mess3 #2 Coloring: PCs 1-2
Mess3 #3 Coloring: PCs 1-2
Tom Quantum #1 Coloring: PCs 1-2
Tom Quantum #2 Coloring: PCs 1-2
Blank
PC1
PC2
Mess3 Label 0
Mess3 Label 1
Mess3 Label 2
TomQ Label 0
TomQ Label 1
TomQ Label 2
TomQ Label 3


3D PCA PCs 3,4,5
Mess3 #1 Coloring: PCs 3-4-5
Mess3 #2 Coloring: PCs 3-4-5
Mess3 #3 Coloring: PCs 3-4-5
Tom Quantum #1 Coloring: PCs 3-4-5
Tom Quantum #2 Coloring: PCs 3-4-5
Blank
PC5
Mess3 Label 0
Mess3 Label 1
Mess3 Label 2
TomQ Label 0
TomQ Label 1
TomQ Label 2
TomQ Label 3

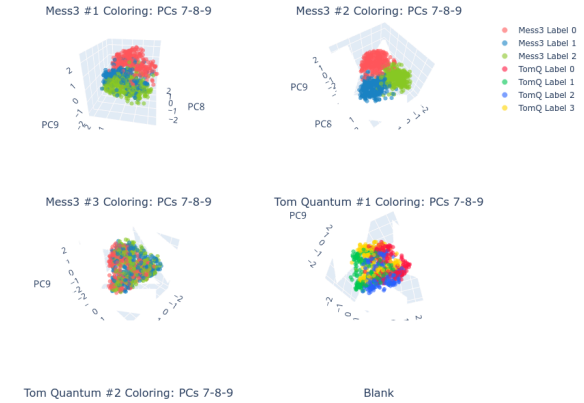

2D PCA PCs 4,5
Mess3 #1 Coloring: PCs 4-5
Mess3 #2 Coloring: PCs 4-5
Mess3 #3 Coloring: PCs 4-5
Tom Quantum #1 Coloring: PCs 4-5
Tom Quantum #2 Coloring: PCs 4-5
Blank
PC4
PC5
Mess3 Label 0
Mess3 Label 1
Mess3 Label 2
TomQ Label 0
TomQ Label 1
TomQ Label 2
TomQ Label 3

3D PCA PCs 6,7,11
Mess3 #1 Coloring: PCs 6-7-11
Mess3 #2 Coloring: PCs 6-7-11
Mess3 #3 Coloring: PCs 6-7-11
Tom Quantum #1 Coloring: PCs 6-7-11
Tom Quantum #2 Coloring: PCs 6-7-11
Blank
PC11
PC6
PC7
Mess3 Label 0
Mess3 Label 1
Mess3 Label 2
TomQ Label 0
TomQ Label 1
TomQ Label 2
TomQ Label 3

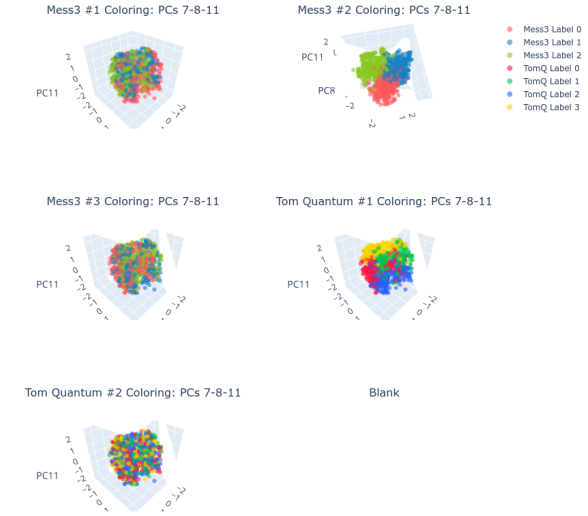
2D PCA PCs 6,11



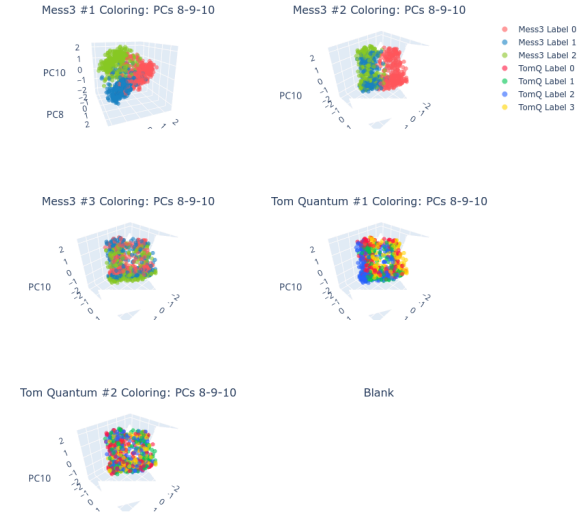
3D PCA PCs 7,8,9



3D PCA PCs 7,8,11

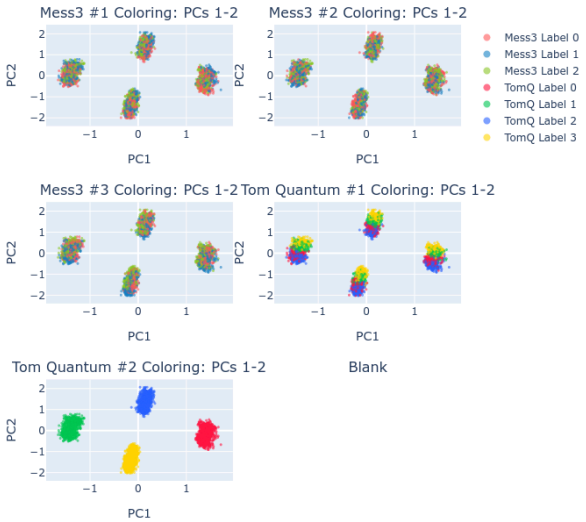


3D PCA PCs 8,9,10

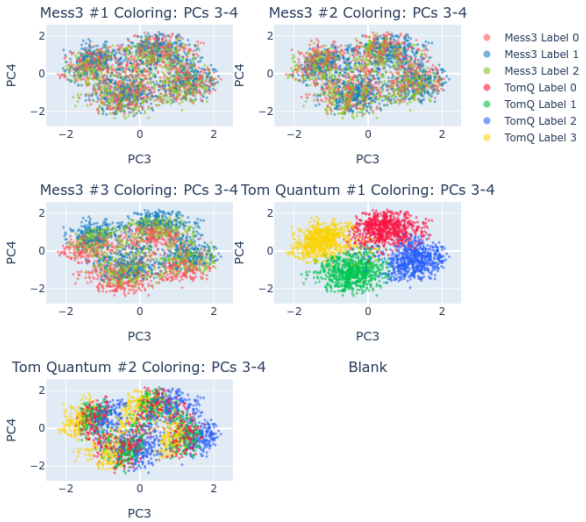


Layer 1

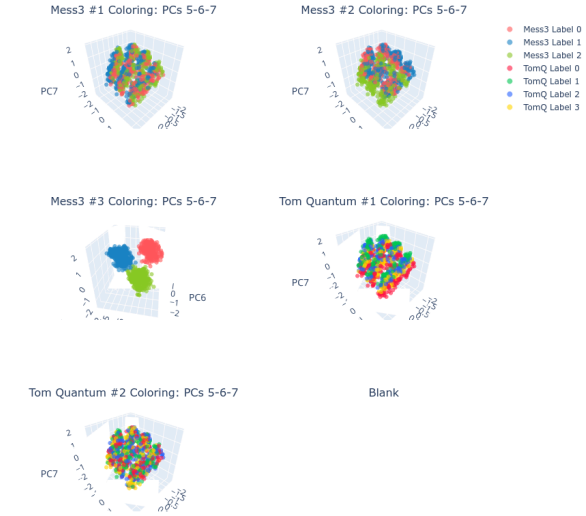
2D PCA PCs 1,2



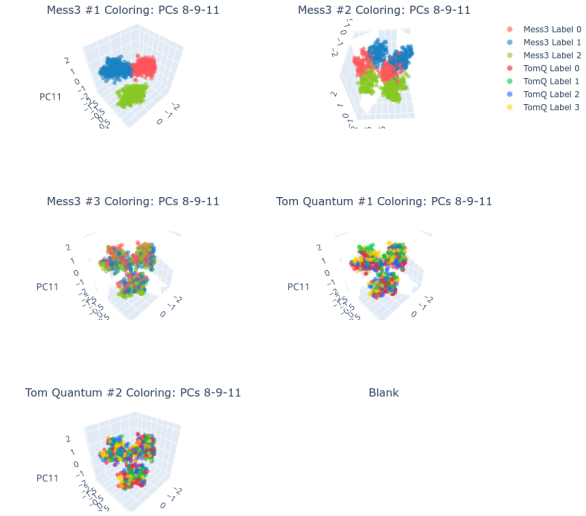
2D PCA PCs 3,4



3D PCA PCs 5,6,7

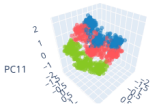


3D PCA PCs 8,9,11



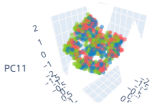
3D PCA PCs 9,10,11

Mess3 #1 Coloring: PCs 9-10-11

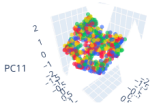


Mess3 #2 Coloring: PCs 9-10-11

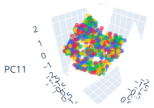
Mess3 #3 Coloring: PCs 9-10-11



Tom Quantum #1 Coloring: PCs 9-10-11



Tom Quantum #2 Coloring: PCs 9-10-11



Blank